

Python Cheatsheet

arithmetische Operatoren

Von der höchsten zur niedrigsten Priorität:

Operator	Operation	Beispiel
**	Exponent	2 ** 3 = 8
%	Modulus/Rest	22 % 8 = 6
//	Integer Division	22 // 8 = 2
/	Division	22 / 8 = 2.75
*	Multiplikation	3 * 3 = 9
-	Subtraktion	5 - 2 = 3
+	Addition	2 + 2 = 4

Datentypen

Datentyp	Beispiel	Beschreibung
int	-2, -1, 0, 1, 2, 3, 4, 5	Ganze Zahlen
float	-1.25, -1.0, -0.5, 0.0, 0.5, 1.0, 1.25	Zahlen mit Dezimalstellen
string	'a', 'Hello!', "Python"	Text und Zeichen
bool	True, False	Wahr oder Falsch-Werte
NoneType	None	Steht für „kein Wert“ oder „nichts“
list	[1, 2, 3], ['a', 'b', 'c']	Geordnete, veränderbare Sammlung
dict	{'name': 'Alice', 'age': 30}	Schlüssel-Wert-Paare
tuple	(1, 2, 3), ('a', 'b')	Geordnete, unveränderliche Sammlungen

user input

Der Benutzer gibt etwas ein. Es wird als String gespeichert.

```
my_name = input('What is your name? ')
print(f"Hi, {my_name}")
```

output

eine Nachricht auf dem Bildschirm ausgeben

```
print("Hello, World!")
```

Variablen

um Daten zu speichern

```
meine_variable = "Hallo"
```

Variablennamegebung die nicht erlaubt sind

```
my variable = 'Hello'
```

```
$$@variable = 'Hello'
```

```
23_var = 'Hello'
```

Variablennamen sollten in Kleinbuchstaben geschrieben werden, wobei Wörter zur Verbesserung der Lesbarkeit gegebenenfalls durch Unterstriche getrennt werden.

Bedingungen

if else, elif

```
if bedingung:
    #Anweisungen, falls bedingung erfüllt ist
elif bedingung2:
    #Anweisungen
else:
    #Anweisungen
```

Vergleichsoperatoren

Diese Operatoren werden je nach den von Ihnen angegebenen Werten als „True“ (Wahr) oder „False“ (Falsch) ausgewertet.

Operator	Bedeutung	Beispiel
<	kleiner als	a < 10
>	größer als	b>4
==	gleich	c=='yes'
<=	kleiner gleich	d<=5
>=	größer gleich	e<=7
!=	ungleich	g!='no'
'in'	in	'x' in 'mexico'
'not in'	nicht in	y not in 'mexico'

Booleanoperatoren

Operator	Ausdruck	Auswertung
and	True and True	True
	True and False	False
	False and True	False
	False and False	False
or	True or True	True
	True or False	True
	False or True	True
	False or False	False
not	not True	False
	not False	True

Iterationen

Wiederholt einen Code-Block mehrmals.

for-loop

Die for-Schleife durchläuft eine Liste, ein Tupel, ein Dictionary, set oder eine Zeichenkette:

```
for item in list:
    #Anweisungen für item
    #Anweisungen für item
#Anweisungen nach der Schleife
```

range()

Die Funktion range() gibt eine Zahlenfolge zurück. Sie beginnt bei 0, erhöht sich um jeweils 1 und endet vor einer bestimmten Zahl:

```
# range(start, stop, step)

for i in range(n):
    #Anweisungen n mal Wiederholen
```

while-loop

wiederholt einen Codeblock, solange eine Bedingung wahr ist.

break beendet die Schleife.

continue beendet den aktuellen Durchlauf.

```
while bedingung:
    #Anweisungen
```

Funktionen

Definiert wiederverwendbare Codeblöcke

```
def funktionsname(Var1, Var2=4):
    #Anweisungen
    #Anweisungen
    return result          #optional
```

Funktionsnamen in **snake_case**

Gute Funktionsnamen machen den Code **lesbarer** und **verständlicher**.

Anstatt Abkürzungen zu verwenden, wähle lieber längere, aussagekräftige Namen.

✅ Gut: `calculate_final_price` oder `send_confirmation_email`
❌ Schlecht: `calc_fp` oder `sce`

Laden eines Moduls

Importanweisung

```
import modulname
```

```
import modul as short
```

```
from modul import *
```

Befehlsaufruf

```
modulname.befehl()
```

```
short.befehl()
```

```
befehl()
```

`dir(modulname)` - zeigt alle Befehle im Modul an
`help(befehl)` - zeigt die Hilfe für den Befehl an

Module

random	zufallszahlen
math	Mathematic
turtle	Turtle-Grafik
os	Betriebssystem
sys	console

Arbeiten mit Listen

<code>len(myList)</code>	Länge von myList
<code>myList[i]</code>	i-tes Element der Liste
<code>myList[i:j]</code>	Ausschnitt von i bis j
<code>x in myList</code>	True wenn x in myList ist
<code>myList.append(x)</code>	x myList anhängen
<code>myList.insert(i,x)</code>	x vor der Stelle i einfügen
<code>myList[i]=x</code>	element i ersetzen
<code>myList.remove(x)</code>	entfernt x aus myList
<code>myList.pop([i])</code>	entfernt das i-te Element
<code>myList=[]</code>	

Zeichenketten (Strings)

<code>str.lower()</code>	in Kleinbuchstaben umwandeln
<code>str.upper()</code>	in Grossbuchstaben umwandeln
<code>str.replace(old,new)</code>	old durch new ersetzen
<code>str.split()</code>	Teilt den String auf
<code>str[1:5]</code>	Zeichen 1-5 anzeigen
<code>list(str)</code>	erzeugt eine Buchstabenliste

Turtlegraphics

<code>fd(s)/forward()</code>	Vorwärts s Einheiten
<code>bk(s)/backward()</code>	Rückwärts s Einheiten
<code>rt(a)/right()</code>	Drehen nach Rechts
<code>lt(a)/left()</code>	Drehen nach Links
<code>pu()/penup()</code>	nicht Zeichnen
<code>pd()/pendown()</code>	ab jetzt Zeichnen
<code>circle(r)</code>	Kreis mit Radius r
<code>fillcolor(f)</code>	Füllfarbe setzen
<code>begin_fill()</code>	ab jetzt Ausfüllen
<code>end_fill()</code>	Ausfüllen beenden
<code>home()</code>	zum Startpunkt
<code>reset()</code>	neu Anfangen
<code>done()</code>	hält das Fenster geöffnet